



BMTD3: An Enhanced TD3 for Mapless Autonomous Navigation with BiGRU and Multi-Head Attention

Haohua Zhang¹ · Ying Sun² · Pengyuan Liu¹ · Derui Ding¹ · Ran Sun¹

Received: 20 May 2025 / Accepted: 11 December 2025
© The Author(s) 2025

Abstract

Despite achieving significant results, mapless navigation still faces enormous challenges because of environmental perception errors caused by noises, uncertainties in sensor data, and the inadequate real-time performance and decision-making capabilities in dynamic environments. This paper proposes a novel deep-reinforcement-learning-based autonomous navigation framework, BMTD3, to address goal-driven navigation tasks in complex dynamic environments, with both static and dynamic obstacles, without pre-built maps. A bidirectional gated recurrent unit is employed to acquire the memory ability for the network and alleviates partly the observability problem. A multi-head attention mechanism is developed to extract different key historical features for policy generation and value estimation in navigation, improving the response ability to complex dynamic scenes, as well as the stability and accuracy in complex environments. Furthermore, a novel reward function is designed to enhance navigation accuracy and adaptability, effectively addressing the issue of reward sparsity. Simulation results validate that, compared to conventional TD3 and other baseline navigation strategies, BMTD3 achieves superior navigation success rates and enhanced robustness across both static and dynamic scenarios.

Keywords Mapless navigation · Deep reinforcement learning · Multi-head attention · Reward functions

1 Introduction

In recent years, autonomous navigation has become one of the core research directions in the field of mobile robotics with the rapid advancement of machine learning, and has been widely applied in various fields, including autonomous driving [1], intelligent transportation [2], service robotics [3],

logistics and transportation [4], exploration and rescue [5], and healthcare [6]. Robotic autonomous navigation refers to the ability of a robot to perceive, decide, and control its movements, enabling it to travel safely and efficiently from its current position to a designated target location without human intervention [7]. The robot must ensure that it does not collide with obstacles during navigation.

Robotic autonomous navigation is essential for task execution. Preconstructed environmental maps are commonly relied upon by traditional autonomous navigation methods in order to reach target locations while avoiding obstacles. The use of Simultaneous Localization and Mapping (SLAM) [8] as a widely used technique to build environmental maps has become a common practice among these methods. SLAM reaches real-time robot localization and map construction by integrating multi-sensor data, including LiDAR, cameras, and inertial measurement units, which forms the basis for subsequent tasks [9]. However, SLAM suffers from high computational complexity and time consumption when vision-based robotic navigation is a concern [10, 11]. When the environment changes over time or the robot enters a new environment, the map must be reconstructed, leading to both the time consumption of map construction and updating, and

✉ Ying Sun
yingsun1991@usst.edu.cn

Haohua Zhang
233370882@st.usst.edu.cn

Pengyuan Liu
233370854@st.usst.edu.cn

Derui Ding
deruiding2010@usst.edu.cn

Ran Sun
sunr1990@usst.edu.cn

¹ Department of Control Science and Engineering, University of Shanghai for Science and Technology, Shanghai 200093, P.R. China

² Business School, University of Shanghai for Science and Technology, Shanghai 200093, P.R. China

the low real-time performance and computational efficiency. Furthermore, map-based methods require prior environmental knowledge, and any environmental changes necessitate map reconstruction, making navigation heavily dependent on the quality of the generated map [12].

To overcome the above challenges, mapless navigation is an effective alternative solution. Unlike traditional map-based methods, it enables environmental perception and decision-making directly from sensor data (such as LiDAR or cameras) without relying on preconstructed maps, making it well-suited for tasks in unknown and dynamic environments [13–15]. Specifically, mapless navigation does not rely on a global map, nor does the navigation policy perform any map-based planning. It only depends on lightweight localization techniques such as odometer, IMU, or UWB to provide approximate pose information. The target position is also represented in relative coordinates, without requiring complete environmental knowledge, thereby enabling navigation in dynamic or complex environments. This approach not only improves adaptability to environmental changes but also significantly reduces computational complexity and storage requirements. However, despite its advantages, it still faces several challenges in practical applications. Specifically, noises and uncertainties of sensor data may introduce errors in environmental perception, affecting the accuracy and robustness of navigation. In highly dynamic environments, robots must quickly respond to moving obstacles, placing higher demands on real-time performance and decision-making capabilities. In recent years, Deep Reinforcement Learning (DRL) [16], as a powerful autonomous decision-making method, has been widely applied to mapless navigation tasks. DRL is a potent combination of Deep Learning (DL) and Reinforcement Learning (RL) [17, 18], which enables robots to learn optimal strategies through interaction with the environment and trial-and-error, thereby achieving specific goals. However, existing DRL-based navigation methods still have limitations, especially in handling high-dimensional inputs and predicting the movement of dynamic obstacles in complex environments.

To address the aforementioned issues, this paper proposes a mapless autonomous navigation framework based on LiDAR and deep reinforcement learning [19, 20]. The Bidirectional Gated Recurrent Unit (BiGRU) network and Multi-Head Attention mechanism are integrated into the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm [21], resulting in a novel DRL model, BMTD3, designed to achieve efficient autonomous navigation in both static and dynamic environments. Specifically, the main contributions of this paper are as follows:

- a novel autonomous navigation framework BMTD3, which combines current and historical observation states, is proposed based on deep reinforcement learning to

address goal-driven navigation in environments with static and dynamic obstacles. It effectively utilizes the LiDAR data collected by robots without relying on pre-built maps;

- the BiGRU is employed to process historical observation sequences. Its bidirectional structure is used to extract historical features, providing memory ability for the navigation network, and effectively alleviating the partial observability problem;
- a multi-head attention mechanism is introduced to adaptively adjust the extraction of key features according to different network objectives. By extracting key information between multiple input features, the response ability to complex dynamic scenes is improved, and the stability and accuracy of the robot in complex environments are enhanced;
- a novel reward function is designed, considering multiple factors affecting the navigation task, including obstacle distances, target distances, and target orientation errors. It effectively addresses the issue of reward sparsity and improves the quality of feedback for the robot's actions during navigation.

Following this introduction, Section 2 reviews related work. Section 3 introduces the preliminaries. Section 4 presents the proposed method. Section 5 describes the experimental simulations and analysis. Finally, Section 6 summarizes the main conclusions.

2 Related Work

This section provides an overview of previous studies related to our proposed approach. It focuses on two key areas: deep reinforcement learning, which forms the foundation for autonomous decision-making, and mapless navigation, which enables robotic systems to operate in unknown environments without relying on pre-constructed maps. These two topics are discussed in the subsequent subsections.

2.1 DRL

Since the introduction of Deep Q-Network (DQN) by the DeepMind team [22, 23], significant progress has been made in the field of Deep Reinforcement Learning (DRL), which integrates deep networks into reinforcement learning to address the curse of dimensionality, enabling agents to directly learn control policies from high-dimensional sensor inputs. Subsequently, Hasselt et al. [24] proposed DDQN, which reduces the overestimation of Q-values by decoupling action selection from action evaluation. Wang et al. [25] introduced Dueling DQN, which improves the accuracy

of Q-value estimation by separating the state value function and the action advantage function. However, DQN typically handles only discrete actions. In 2016, to address the limitations of discrete actions, Lillicrap et al. [26] proposed Deep Deterministic Policy Gradient (DDPG), which combines the advantages of DQN and policy gradient methods, using deep neural networks to approximate both the policy and Q-value functions for continuous actions. Later, Tai et al. [27] proposed Asynchronous Deep Deterministic Policy Gradient (ADDPG), which improves training speed and robustness in complex map navigation. Mnih et al. [28] introduced A3C, which improves sample efficiency by asynchronously training multiple agents in parallel. Schulman et al. [29] proposed PPO, which enhances stability and sample efficiency by limiting the extent of policy updates. To solve the Q-value overestimation problem in DDPG, Fujimoto et al. [21] introduced TD3, which improves the stability and performance of the algorithm through double Q-learning and delayed updates. Haarnoja et al. [30] proposed SAC, which improves exploration and robustness by maximizing the entropy of the policy. Hausknecht et al. [31] incorporated Recurrent Neural Networks (RNN) into DRL to handle temporal dependencies in partially observable environments. Vaswani et al. [32] introduced the attention mechanism into DRL to improve the handling of high-dimensional state spaces.

2.2 Mapless Navigation

In recent years, DRL has been widely applied to collision avoidance and navigation in robots within unknown environments [33–44]. For mapless navigation under different LiDAR configurations, Zhang et al. [33] introduced a DRL method based on support points, which can efficiently handle various obstacles and crowded scenes, demonstrating good generalization ability. Shi et al. [34] proposed an asynchronous DDPG algorithm for mapless end-to-end navigation using sparse LiDAR data in simple static scenes. To address the issue of mapless navigation for moving robots in dynamic environments, Jin et al. [35] proposed a 2D LiDAR-based mapless navigation method, which incorporates self-safety and social safety awareness, and employs RL-trained strategies to efficiently avoid obstacles in dynamic environments. Zhang et al. [36] proposed a new input preprocessing method, IPAPRec, which optimizes LiDAR data representation by adapting the inverse function of parameters, thus improving the generalization performance and learning speed of DRL navigation agents. This method is validated through both simulation and real-world experiments. To effectively handle time-series data and capture useful information, Zhu et al. [37] introduced a DRL and LSTM-based robot navigation framework, which uses sequence-normalized LiDAR scanning (LNDNL) to perceive dynamic environments and accelerates training via a cus-

tom LiDAR integration simulator, thereby improving robot navigation capabilities in dynamic environments. Everett et al. [38] proposed a dynamic obstacle avoidance algorithm, GA3C-CADRL, which processes the observation information of agents in the environment through an LSTM network. Xue et al. [39] proposed a navigation framework called RLPlanNav that uses an LSTM-based RDPG algorithm to handle environment perception in order to generate smooth and dynamically feasible trajectories. Zhu et al. [40] proposed an autonomous navigation system based on a Dreamer model, which combines RNNs to construct a latent space dynamic model to realize long-term trajectory prediction and policy optimization. For high-dimensional and continuous navigation tasks of controlled robots, in order to improve computational efficiency and training convergence speed, and ensure that the model pays attention to the most important historical data, the attention mechanism is introduced. Han et al. [41] proposed a deep reinforcement learning method combining a self-state attention mechanism and multi-sensor fusion to improve the obstacle avoidance ability of robots in complex dynamic environments. Compared to LSTM, GRU has lower computational complexity, reduced storage and computational overhead, and more stable training. Yang et al. [42] proposed a spiking neural network framework based on Multi-Critic Deep Deterministic Policy Gradient (MC-DDPGSM), which integrates a Spiking-GRU layer and a multi-critic network to address the partial observability problem in mapless navigation and demonstrate higher navigation success rates, shorter paths, and lower energy consumption in complex dynamic environments. Böhm et al. [43] introduced a feature extraction method based on GRU, significantly improving DRL training efficiency and strategy performance on real robot platforms, particularly in complex dynamic environments.

To enable robots to remember past observations, filter key historical data, reduce computational complexity, and improve the stability and real-time performance of robot navigation, Jia et al. [44] proposed a GRU-Attention-based TD3 network for goal-directed navigation of mobile robots in unknown environments. However, GRU can only propagate information from the past to the present, limiting its ability to model long-term dependencies and potentially neglecting distant information. Additionally, single-head attention lacks the ability to extract multi-level features effectively. To this end, a BMTD3 method is proposed to enable mobile robots to navigate quickly and safely in complex and dynamic environments.

3 Preliminaries

This paper focuses on a differential wheeled robot, whose model is shown in Fig. 1. The robot's pose is described by

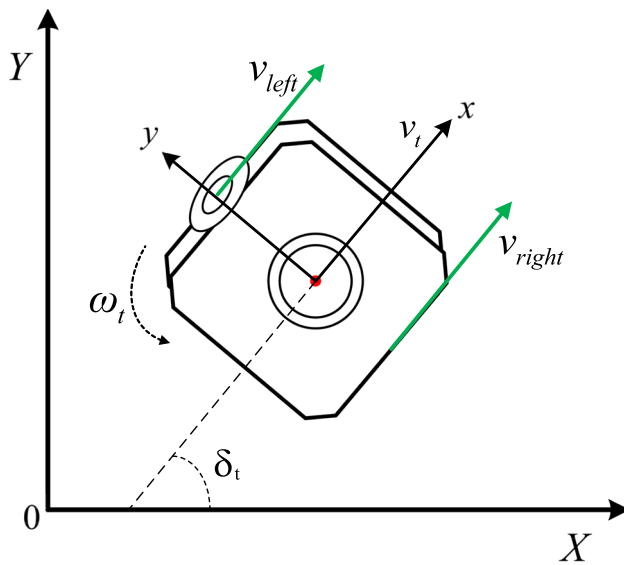


Fig. 1 The robot model considered in our research

its position and movement direction in space. Specifically, the robot's linear velocity at the current state is denoted as v_t , and the angular velocity is denoted as ω_t . Its kinematic model is given by the following equations

$$\begin{cases} \dot{X}_t = v_t \cos \delta_t \\ \dot{Y}_t = v_t \sin \delta_t \\ \dot{\delta}_t = \omega_t \end{cases} \quad (1)$$

where v_t and ω_t are the control inputs of the robot. During the navigation process, the hardware constraints of the robot need to be considered. In practical operations, the robot does not have a negative linear velocity, and neither linear nor angular velocities can increase indefinitely. As such, based on the robot's actual driving capabilities, the minimum linear velocity is set as 0, the maximum linear velocity is $v_{\max}$, and the maximum angular velocity is denoted as $\omega_{\max}$, that is, $v_t \in [0, v_{\max}]$ and $\omega_t \in [-\omega_{\max}, \omega_{\max}]$.

In robot LiDAR navigation, the famous Markov Decision Process (MDP), which relies on the current state to make decisions, is suitable for static or deterministic environments [45]. In practical applications, sensor measurements are usually affected by noise, occlusions, and limited fields of view, making them difficult for traditional fully observable MDPs to effectively model uncertainties. Fortunately, the partially observable MDP (denoted as POMDP) can adapt to dynamic environments through probabilistic modeling, handle observation noises, and infer the optimal strategy by incorporating historical information for reliable navigation and obstacle avoidance, thereby improving navigation robustness.

Based on an employed 2D LiDAR sensor, the POMDP model is expressed as $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O})$, where $\mathcal{S}$ rep-

resents the state space of the robot, $\mathcal{A} = \{a_0, \dots, a_t\}$ denotes the action space, $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ represents the state transition probability, $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, Ω denotes the observation set, and $\mathcal{O} : \mathcal{S} \times \mathcal{A} \times \Omega \rightarrow [0, 1]$ means the conditional observation transition probability. At time step t , the environmental state is $s_t \in \mathcal{S}$ and the action taken by the robot is $a_t = \{v_t, \omega_t\} \in \mathcal{A}$. The environmental state will be transferred to the next state s_{t+1} according to the state transition probability $\mathcal{P}(s_{t+1} | s_t, a_t)$. The agent receives a new observation $\mathcal{O}(z_t | s_{t+1}, a_t)$ and simultaneously obtains a reward R_t . It should be pointed out that the state space $\mathcal{S}$ can only be obtained by the global observer and hence the observation space $\Omega = \{z_0, z_1, \dots, z_t\}$, where $z_t = \{z_t^{\text{lidar}}, z_t^g, z_t^{\text{velocity}}\}$ is available. In the simulation environment, the surrounding environmental information of the robot is abstracted into 40 uniformly spaced sector regions within the range of -180° to 180° , which serves as one of the observation inputs, denoted as $z_t^{\text{lidar}} = \{z_t^{\text{lidar}1}, \dots, z_t^{\text{lidar}40}\}$. Furthermore, it is assumed that the target position and the robot's position are always known. The relative distance d_t^g between the robot and the target can be calculated in real time by using the Euclidean distance formula, while the angular difference between the robot's current direction and the direction towards the target position is denoted as φ_t^g , forming $z_t^g = \{d_t^g, \varphi_t^g\}$. Finally, the linear velocity v_t and yaw angular velocity ω_t form $z_t^{\text{velocity}} = \{v_t, \omega_t\}$.

By means of interacting with the unknown environment, the robot tries to maximize the received reward $\mathcal{G}_t$ based on the observation $z_t \in \Omega$ through an optimal policy π^*

$$\mathcal{G}_t = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \quad (2)$$

where $\gamma \in (0, 1]$ is a discount factor. In summary, the objective of this study is to develop a DRL-based mapless autonomous navigation method to address goal-driven navigation in environments with static and dynamic obstacles.

In this paper, the navigation framework is illustrated in Fig. 2 for a mapless navigation task. The overall framework of BMTD3-based mapless navigation consists of three main components: the environment state input, the agent, and the BMTD3 network. The state input includes real-time sensor data, such as the agent's velocity, LiDAR scan data, and the relative position and orientation with respect to the goal. These observations are fed into the BMTD3 network, which employs a BiGRU module to process historical sequences, capture temporal dependencies, and incorporate memory of past observations, thereby addressing the partial observability problem in mapless scenarios. A multi-head attention mechanism is used to dynamically assign weights to historical features, extract key information, and help the network

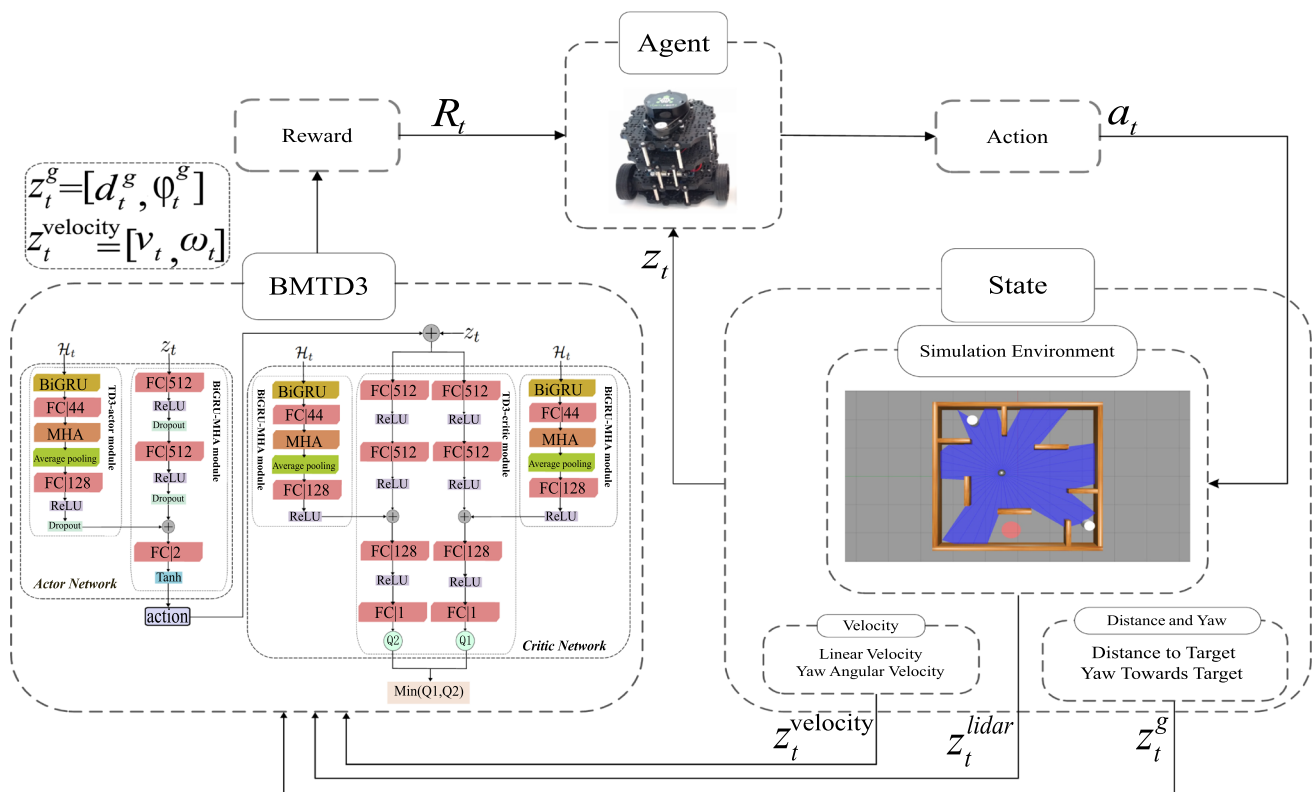


Fig. 2 The overview of navigation framework

focus on critical states. The actor network outputs continuous velocity commands for the agent, while the critic network evaluates action-value pairs to guide the learning process. This closed-loop interaction enables the agent to perceive, reason, and act directly from raw sensory inputs, without relying on any preconstructed map.

4 Methodology

To achieve mapless autonomous navigation and obstacle avoidance, this paper proposes an improved TD3 algorithm, named BiGRU-Multi-Head-Attention-TD3 (i.e. BMTD3). In comparison with traditional TD3 structures, our BMTD3 network still follows the classical actor-critic architecture, with improved actor and critic networks by integrating novel BiGRU-Multi-Head-Attention (i.e., BiGRU-MHA) modules (see Fig. 3 for more details). Specifically, the BiGRU-MHA module, which combines the BiGRU with the Multi-Head Attention (MHA) mechanism, can effectively utilize historical observation sequences, extract multi-level features in partially observable environments, enhance the capability of perceiving and responding to changes in dynamic obstacle environments, and improve the stability and generalization of the policy. The BMTD3 network adopts the standard TD3 as its backbone and integrates BiGRU and

MHA into the actor and critic networks of the standard TD3, forming new actor and critic structures. As a result, the actor network is composed of the TD3-actor module and the BiGRU-MHA module, while the critic network consists of the TD3-critic module and the BiGRU-MHA module. The TD3-actor module is responsible for learning the features of the current observation state, providing basic policy learning capabilities, and optimizing based on the feedback from the environment. The TD3-critic module is responsible for evaluating the value of the current action and providing feedback signals to the actor to improve the policy.

To address the challenges of mapless navigation, our approach does not rely on any preconstructed global maps. Instead, it utilizes real-time sensory inputs such as LiDAR scans, the robot's velocity, and the relative position and orientation of the target. These inputs are fed into the BMTD3 network, where a BiGRU module captures temporal dependencies, and a multi-head attention mechanism extracts critical historical features to support decision-making. The robot's localization is provided by a lightweight positioning system odometry. This design enables the agent to operate effectively in unknown and dynamic environments. Furthermore, after executing each action, the agent receives feedback from the environment in the form of a new observation and a scalar reward. This feedback is processed through the critic networks to compute temporal-difference (TD) error, which

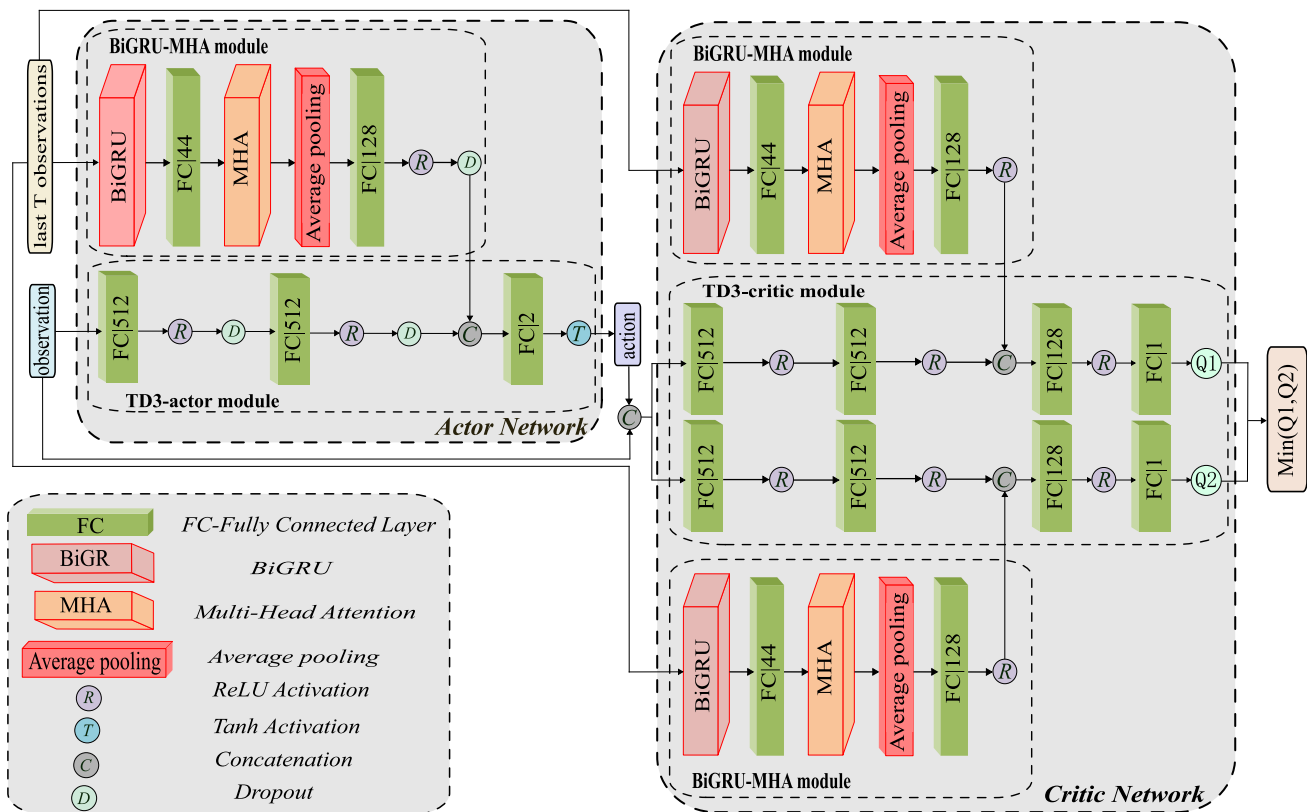


Fig. 3 The BMTD3 algorithm

guides the parameter updates of both the actor and critic networks, thus forming a closed-loop learning process.

4.1 BiGRU-MHA Modules

The traditional TD3 network only uses the current observation state as inputs, which cannot effectively leverage historical information. Therefore, this paper constructs three BiGRU-MHA modules combined with original TD3-actor and TD3-critic modules to generate the improved actor and critic networks, respectively. These three BiGRU-MHA modules with no parameter sharing are independent of each other and receive the same historical observation sequence $\mathcal{H}_t = \{z_{t-T}, z_{t-T+1}, \dots, z_{t-1}\} \in \mathbb{R}^{T \times D}$ from the past T time steps. Furthermore, their goals and gradient sources are also different. The BiGRU-MHA module in the actor network focuses on extracting policy-relevant temporal features and is updated through gradients from the policy loss. The two critic networks each employ their own independent BiGRU-MHA modules to encode historical context information for state-action value estimation. The outputs of these modules are concatenated with the current state and action, and passed through separate Q-value estimators. These two BiGRU-MHA modules are optimized based on the temporal-difference (TD) learning objective. The utilization of this

kind of independent BiGRU-MHA modules provides greater flexibility and representation ability. Additionally, considering the batch dimension, $\mathcal{H}_t$ should belong to $\mathbb{R}^{B \times T \times D}$ during training. Specifically, B denotes the batch size and D indicates the feature dimension and their values are, respectively, 128 and 44 in our subsequent experiments.

In the actor network, the BiGRU-MHA module focuses on which features in the historical observation sequence can be utilized to improve the quality of the current action selection, helping the policy network to make more reasonable control behavior. In navigation tasks, because of LiDAR's limited sensing range, the robot may overlook information such as its recent movements and the distance to the target position, leading to incomplete state observations. Therefore, it's necessary to leverage historical observation data to compensate for the limitations of the current observation. This helps the robot infer the hidden states of the environment and make better decisions. Compared to GRU, LSTM, Transformer and CNN, the BiGRU has fewer parameters, is more lightweight, converges faster, and has lower computational cost. Furthermore, such a module with bidirectional architectures enables the extraction of historical features from complete historical sequences to generate a context-rich hidden observation states $h_t^a = [\overrightarrow{h}_t^a, \overleftarrow{h}_t^a]$, alleviating the partial observability problem and enhancing the robot's perception of dynamic

environmental changes. The output h_t^a of the BiGRU in the actor network is denoted as follows

$$h_t^a = BiGRU(\mathcal{H}_t) \in \mathbb{R}^{B \times T \times D} \quad (3)$$

which contains both forward and backward hidden observation $\vec{h}_t^a$ and $\overleftarrow{h}_t^a$

$$\vec{h}_t^a = GRU(z_t, \vec{h}_{t-1}^a), \quad \overleftarrow{h}_t^a = GRU(z_t, \overleftarrow{h}_{t+1}^a). \quad (4)$$

To maximize the extraction of temporal structural information, the BiGRU is placed at the first layer in BiGRU-MHA modules for processing historical observation data. It not only avoids the loss of historical information caused by prior compression but also leads to faster convergence and more stable training performance. To make better use of historical features and facilitate the processing and use of the later network, an FC layer is added after the BiGRU, compressing its output dimension by half to match the fixed dimension required by the MHA mechanism. The corresponding output $\tilde{h}_t^a$ can be formulated as

$$\tilde{h}_t^a = W_1^a h_t^a + b_1^a \quad (5)$$

where W_1^a is the weight matrix, and b_1^a is the bias term. In the navigation task, compared to the standard attention mechanism, the MHA mechanism can process multiple distinct feature subspaces in parallel, enabling the extraction of key features from historical data such as laser scans, target positions, robot positions and velocities. The MHA mechanism extracts the key historical features that has the greatest impact on the decision. Therefore, applying the MHA obtains

$$u^a = MHA(\tilde{h}_t^a) = \text{Concat}(\text{head}_1, \dots, \text{head}_{\tilde{h}}) W^O \quad (6)$$

with

$$\text{head}_s = \text{softmax} \left(\frac{Q_s K_s^T}{\sqrt{d_k}} \right) V_s, \quad 1 \leq s \leq \tilde{h} \quad (7)$$

and

$$Q_s = \tilde{h}_t^a W_s^Q, \quad K_s = \tilde{h}_t^a W_s^K, \quad V_s = \tilde{h}_t^a W_s^V \quad (8)$$

where $\tilde{h}$ is the total number of attention heads; W^O is the output transformation matrix; head_s represents the output of the s -th head; and W_s^Q , W_s^K and W_s^V are the query, key and value transformation matrices of the s -th head, respectively. Q_s , K_s and V_s represent the query, key and value for the s -th head in the MHA mechanism respectively. d_k denotes the dimension of each head.

To achieve dimensionality reduction and feature compression, enhance the global representation of historical features,

and ensure navigation stability, the MHA applies average pooling to receive

$$\bar{h}_t^a = \frac{1}{T} \sum_{t-T}^{t-1} u_t^a \in \mathbb{R}^{B \times D} \quad (9)$$

where u_t^a denotes the attention output at time step t . To further integrate key information and filter out irrelevant data, an FC layer is also used to compress and transform the information while unifying the dimensions. Then, the ReLU activation function is employed to introduce non-linearity, and the dropout is utilized to prevent overfitting. The final output is represented as follows

$$f_a^{his} = \delta \left(\sigma \left(W_2^a \bar{h}_t^a + b_2^a \right) \right) \in \mathbb{R}^{B \times D} \quad (10)$$

where σ represents the ReLU activation function, δ means the dropout, W_2^a is the weight matrix, and b_2^a is the bias term.

In the critic network, the designed BiGRU-MHA modules focus on which features in the historical observation sequence are more strongly correlated with the current value estimation. These two modules independently process the historical observation sequences to avoid gradient explosion or vanishing gradients. It should be pointed out that such modules focus on key historical features that favor long-term returns, although they are similar to those in the actor network. Furthermore, to ensure accurate Q -value estimation, the dropout is not applied, and the formulas are expressed as

$$h_t^{c_m} = BiGRU(\mathcal{H}_t) \in \mathbb{R}^{B \times T \times D} \quad (11)$$

$$\tilde{h}_t^{c_m} = W_1^{c_m} h_t^{c_m} + b_1^{c_m} \quad (12)$$

$$u^{c_m} = MHA(\tilde{h}_t^{c_m}) \quad (13)$$

$$\bar{h}_t^{c_m} = \frac{1}{T} \sum_{t-T}^{t-1} u_t^{c_m} \in \mathbb{R}^{B \times D} \quad (14)$$

where $W_1^{c_1}$ and $W_1^{c_2}$ are the weight matrixes, and $b_1^{c_1}$ and $b_1^{c_2}$ are the bias terms. c_m denotes two networks within the critic, where $m \in \{1, 2\}$. The outputs of the two BiGRU-MHA modules in the critic network are finally given by

$$f_{c_m}^{his} = \sigma \left(W_2^{c_m} \bar{h}_t^{c_m} + b_2^{c_m} \right) \in \mathbb{R}^{B \times D} \quad (15)$$

where $W_2^{c_1}$ and $W_2^{c_2}$ are the weight matrixes, and $b_2^{c_1}$ and $b_2^{c_2}$ are the bias terms.

In summary, these designed BiGRU-MHA modules possess task adaptability, enabling them to autonomously adjust their feature extraction strategies based on the distinct objectives of each network. Furthermore, they facilitate policy generation and value estimation, thereby enhancing the robot's goal-driven and mapless navigation capabilities in complex environments. In both the actor and critic networks,

independent BiGRU-MHA networks are used to avoid issues like gradient explosion or vanishing gradients.

4.2 TD3-Actor/Critic Modules Injected BiGRU-MHA Outputs

The primary objective of the actor network is to learn a policy that generates the optimal action a_t based on the current observation state z_t and the historical information $\mathcal{H}_t$. The position where historical features are concatenated with current features significantly affects the expressiveness and generalization ability of the navigation policy. As such, the choice of concatenation position is nontrivial in the TD3-actor module. As we know, the first FC layer is primarily responsible for initial dimensional expansion and activation, while the second FC layer performs deep feature extraction on the current observation. If the concatenation is performed in deep layers, it may result in information misalignment or loss. Preliminary experiments indicate that concatenating after the second FC layer ensures stable training and semantically rich feature representations, whereas alternative positions often lead to stagnation or failure in navigation. Specifically, the historical features f_a^{his} and the outputs f_a^{cur} of the first two layers (i.e. current observation state feature) are concatenated and passed through a linear layer to generate the action a_t

$$a_t = \tanh(W_5^a [f_a^{his}, f_a^{cur}] + b_5^a) \quad (16)$$

with

$$f_a^{cur} = \delta(\sigma(W_4^a \cdot \delta(\sigma(W_3^a z_t + b_3^a)) + b_4^a)) \in \mathbb{R}^{B \times D}$$

where W_3^a , W_4^a and W_5^a represent the weights, b_3^a , b_4^a , and b_5^a are the bias terms, and the activation function $\tanh$ is used to limit the output within the range of the action space.

The critic network is used to evaluate the value of the current state-action pair. To reduce the overestimation bias, TD3 employs two Q -networks, $Q_1(z_t, \mathcal{H}_t)$ and $Q_2(z_t, \mathcal{H}_t)$, and updates the policy by minimizing the estimated values of these two Q -networks. In the TD3-critic module, the current observation state z_t and the current action a_t are first concatenated, and then passed through two FC layers for feature extraction. The concatenation position is similar to that in the actor network. However, the critic network generally does not use dropout for higher accuracy and low fluctuating Q values, and the concatenation is merely a simple stacking of information and lacks interaction [21]. To learn high-order interaction relationships between features from different sources, two FC layers are employed to map the fused feature vector into scalar values

$$Q_m = W_6^{c_m} \cdot \sigma(W_5^{c_m} [f_{c_m}^{his}, f_{c_m}^{cur}] + b_5^{c_m}) + b_6^{c_m}, \quad m \in \{1, 2\} \quad (17)$$

with the outputs of the first two feature extraction layers

$$f_c^{curm} = \sigma(W_4^{c_m} \cdot \sigma(W_3^{c_m} [z_t, a_t] + b_3^{c_m}) + b_4^{c_m}) \in \mathbb{R}^{B \times D}, \quad m \in \{1, 2\} \quad (18)$$

where $W_3^{c_m}$, $W_4^{c_m}$, $W_5^{c_m}$ and $W_6^{c_m}$ are the weight matrix, and $b_3^{c_m}$, $b_4^{c_m}$, $b_5^{c_m}$ and $b_6^{c_m}$ are the bias terms.

The pseudocode for BMTD3, incorporating the above descriptions, is shown in Algorithm 1. Both the actor and critic networks are updated using an AdamW optimizer. The Q_{target} is used to stabilize training and reduce the overestimation bias of the Q function. where λ is the discount factor and *done* is the termination flag. *done* = 1 indicates the end of the current episode. The loss function of the critic network minimizes the mean squared error (MSE). The critic network updates its parameters through gradient descent to minimize the TD error. The actor is optimized by maximizing the Q -value estimated by the critic. θ_1 and θ_2 represent the parameters of the current critic network, and ϕ represents the parameters of the current actor network. The parameters for the critic and actor networks are updated as follows

$$\theta_1 \leftarrow \theta_1 - \alpha \nabla_{\theta_1} \mathcal{L}_{\text{critic}} \quad (19)$$

$$\theta_2 \leftarrow \theta_2 - \alpha \nabla_{\theta_2} \mathcal{L}_{\text{critic}} \quad (20)$$

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} \mathcal{L}_{\text{actor}} \quad (21)$$

with

$$\begin{aligned} \mathcal{L}_{\text{critic}} &= E[(Q_1 - Q_{\text{target}})^2] + E[(Q_2 - Q_{\text{target}})^2], \\ \mathcal{L}_{\text{actor}} &= -E[Q_1], \end{aligned}$$

where α denotes the learning rate. $\mathcal{L}_{\text{actor}}$ and $\mathcal{L}_{\text{critic}}$ are the loss functions for the actor and critic, respectively. The target network parameters are updated gradually through a soft update mechanism to ensure training stability. The soft update of the target networks is as follows

$$\theta_1' \leftarrow \tau \theta_1 + (1 - \tau) \theta_1' \quad (22)$$

$$\theta_2' \leftarrow \tau \theta_2 + (1 - \tau) \theta_2' \quad (23)$$

$$\phi' \leftarrow \tau \phi + (1 - \tau) \phi' \quad (24)$$

where ϕ' , θ_1' and θ_2' correspond to the parameters of the target networks. τ denotes the soft update coefficient.

4.3 Reward Functions

In DRL, the total reward R_t at each step is composed of multiple sub-reward terms, which are designed to encourage the robot to avoid static and dynamic obstacles while moving toward the target. In this study, the proposed reward function includes target yaw angle reward r_{yaw} , target distance reward r_{dis} , obstacle avoidance reward r_{obs} , linear velocity reward

Algorithm 1 BMTD3 Network.

Initialize the actor network π and the critic networks $Q_1(z_t, \mathcal{H}_t)$ and $Q_2(z_t, \mathcal{H}_t)$ with parameters ϕ, θ_1 and θ_2 . Initialize the target actor network π' and the target critic networks $Q'_1(z_{t+1}, \mathcal{H}_{t+1})$ and $Q'_2(z_{t+1}, \mathcal{H}_{t+1})$ with parameters ϕ', θ'_1 and θ'_2 . Initialize the replay buffer $\mathcal{B}$.

for each episode **do**

INTERACTING:

Initialize the observation state z_1 and the historical observation sequence $\mathcal{H}_1 \leftarrow 0$.

for each time step t **do**

if $t < \text{start_steps}$ **then**

Sample an action randomly: $a_t \leftarrow \text{clip}(\xi, -1, 1), \xi \sim \mathcal{U}(-1, 1)$.

else

Select the action $a_t = \text{clip}(\pi(z_t, \mathcal{H}_t) + \mu, -1, 1), \mu \sim \mathcal{N}(0, OU)$.

end if

Execute a_t to receive the reward R_t , the next observation z_{t+1} and the variable $done_t$.

Append the observation and action to history: $\mathcal{H}_{t+1} \leftarrow \mathcal{H}_t, z_t$.

Update observation $z_t \leftarrow z_{t+1}$ and history $\mathcal{H}_t \leftarrow \mathcal{H}_{t+1}$.

Store $(z_t, a_t, R_t, z_{t+1}, done_t)$ in $\mathcal{B}$.

TRAIN:

Sample $(z_t, a_t, R_t, z_{t+1}, \mathcal{H}_t, done_t)$ from $\mathcal{B}$.

Generate the target action:

$a' = \text{clip}(\pi'(z_{t+1}, \mathcal{H}_{t+1}) + \mu, -1, 1), \mu \sim \mathcal{N}(0, OU)$

Compute the target Q-value:

$Q_{\text{target}} = R_t + \lambda(1 - done_t) \min(Q'_1, Q'_2)$

Get the critic loss function:

$\mathcal{L}_{\text{critic}} = E[(Q_1 - Q_{\text{target}})^2] + E[(Q_2 - Q_{\text{target}})^2]$

Update the parameters θ_1 and θ_2 of critic network.

if $\text{step \% policy_freq} == 0$ **then**

Get the actor loss function: $\mathcal{L}_{\text{actor}} = -E[Q_1]$.

Update the parameter ϕ of actor networks.

Soft-update the parameters θ'_1, θ'_2 , and ϕ' of target networks.

end if

end for

end for

r_v , angular velocity reward r_ω , and additional reward r_2 for task success or failure. This effectively addresses the issue of reward sparsity and improves the quality of feedback for the robot's actions during navigation. R_t is expressed as

$$R_t = r_1 + r_2, \quad (25)$$

where r_1 and r_2 are represented as

$$r_1 = r_{\text{yaw}} + r_{\text{dis}} + r_{\text{obs}} + r_v + r_\omega - 1 \quad (26)$$

$$r_2 = \begin{cases} r_{\text{reach}}, & \text{success} \\ r_{\text{collide}}, & \text{collide} \\ 0, & \text{otherwise.} \end{cases} \quad (27)$$

It should be pointed out that, in reward item r_1 , a fixed negative reward “−1” is added to improve training stability, prevent situations with no rewards, and avoid the robot getting stuck. If this penalty is absent, the agent may learn suboptimal behaviors such as remaining stationary or circling without making progress toward the goal. The constant

penalty encourages the agent to keep exploring. In reward item r_2 , the robot receives the maximum positive reward r_{reach} if it reaches the target position; receives the maximum negative reward r_{collide} if it collides with an obstacle; and otherwise receives the zero reward.

To encourage the robot to face the target as much as possible, an inverse reward is applied to the yaw angle, and the robot's target yaw angle reward r_{yaw} is expressed as

$$r_{\text{yaw}} = -|\varphi_t^g|. \quad (28)$$

To encourage the robot to move closer to the target position and effectively address the sparse reward problem, the target distance reward is calculated based on the robot's current Euclidean distance to the target d_t^g and the initial Euclidean distance to the target d_{initial} . The target distance reward r_{dis} is given by

$$r_{\text{dis}} = \frac{2d_{\text{initial}}}{d_{\text{initial}} + d_t^g} - 1. \quad (29)$$

When the sensor detects that the distance between the robot and an obstacle is smaller than the minimum safe distance d_{min} , an additional penalty is applied. If the distance is greater than or equal to d_{min} , the reward is zero.

$$r_{\text{obs}} = \begin{cases} -20, & d_{\text{min}} < 0.22 \\ 0, & \text{otherwise} \end{cases} \quad (30)$$

The robot's linear velocity directly affects its movement efficiency. A speed that is too fast may make it difficult for the robot to avoid obstacles, while a speed that is too slow may reduce task completion efficiency. To encourage the robot to adopt a reasonable linear velocity for movement, an ideal linear velocity is set, and the linear velocity reward is calculated based on the deviation between the current velocity and the ideal velocity

$$r_v = -((v_{\text{ideal}} - v_t) \times 10)^2 \quad (31)$$

where v_t is the robot's current linear velocity, and v_{ideal} is the ideal linear velocity. The robot's angular velocity ω_t affects its steering ability. However, excessive angular velocity may lead to oscillations. To prevent the robot from using excessively large angular velocities and making sharp turns, a negative reward is given to the square of the angular velocity. Angular velocity reward r_ω is represented as

$$r_\omega = -\omega_t^2 \quad (32)$$

where ω_t is the robot's current angular velocity.

5 Experimental Simulation and Analysis

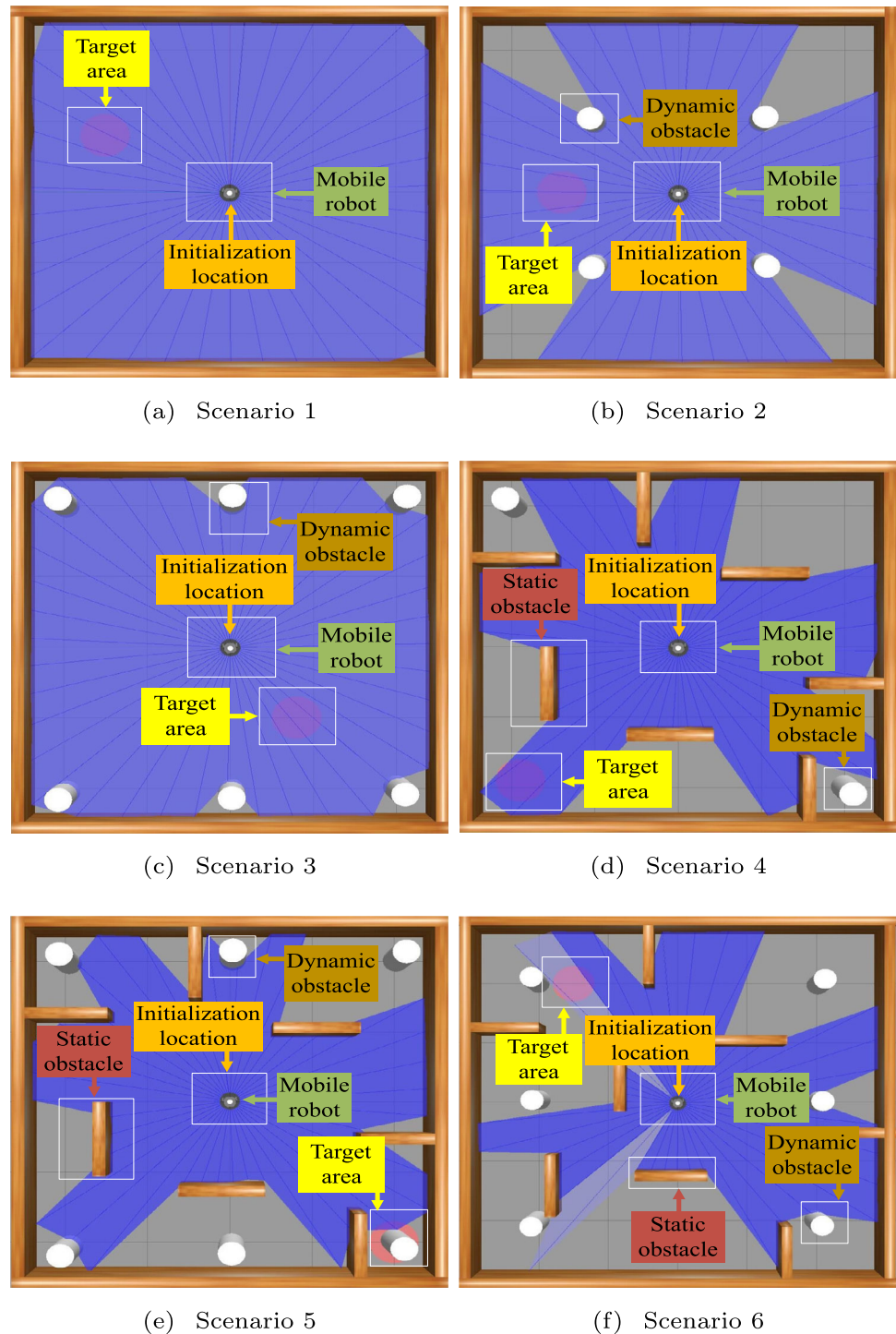
This section presents the experimental evaluation of the proposed BMTD3 framework through simulation-based studies. First, the simulation environment and experimental setup are introduced, followed by a detailed analysis of the results under various scenarios. These experiments are designed to

validate the effectiveness, stability, and adaptability of our approach in dynamic navigation tasks.

5.1 Simulation Setup

In this study, to validate the performance of the developed BMTD3 algorithm, the robot needs to explore the optimal

Fig. 4 Six different training scenarios. The blue color indicates the LIDAR scan. (a) No obstacles; (b) the four obstacles rotate counterclockwise around the initial position of the robot at a constant speed; (c) six dynamically moving random obstacles; (d) static obstacles and two randomly moving dynamic obstacles; (e) and (f) static obstacles and six randomly moving dynamic obstacles



strategy through trial and error. Training in real-world environments is time-consuming, and there may be potential risks associated with robot-environment interactions, such as equipment damage, pedestrian collisions, and environmental destruction, which could lead to socioeconomic losses. To avoid these issues, virtual environments are used for model training. The simulation environment is built by using the Ubuntu 20.04 LTS operating system, integrated with ROS2 Foxy Fitzroy and Gazebo 11.0. The robot model is a TurtleBot3 differential drive robot, equipped with a LiDAR sensor to perceive the environment in a 360-degree range, with a maximum perception distance 3.5 meters and 40 distance data points obtained in each scan. The training platform is a computer equipped with an Intel(R) Xeon(R) CPU E5-1620 v3 @ 3.50GHz, an NVIDIA GeForce GTX 1080 GPU, and 16.0 GB of RAM. Necessary dependencies, including PyTorch (version 1.10.0), are installed to support the RL algorithm. The simulation environment is configured with obstacles and a target to enable TurtleBot3 navigation.

The training environment is a $4.2m \times 4.2m$ rectangular area. The robot starting position is located at the center of the environment (0, 0), and is trained in a series of simulations across six different scenarios, that is, no obstacles, four obstacles rotating around the initial position of the robot, dynamically moving random obstacles, as well as hybrid obstacles, see sub-figures from Fig. 4a to f, where gray black discs mean dynamical obstacles, rectangular blocks in natural wood color stand for fixed obstacles, and the red disc represents the target area.

The training will perform a series of episodes to achieve the algorithm stability, and the training time is the same. By doing so, the target position is randomly generated at subsequent episodes for all scenarios, except the initial training episode. The target is regarded as to be reached if the distance between the target position and the robot's estimated position is less than 0.20m; a collision occurs if the robot comes within 0.13m of an obstacle; and the training is considered as a timeout if the robot does not reach the target within 50 seconds. The episode ends when either the robot reaches the target, a collision occurs, or a timeout happens. Furthermore, the maximum linear velocity and maximum angular velocity are, respectively, set as $v_{max} = 0.22m/s$ and $\omega_{max} = 2.0rad/s$ and the other hyperparameters used during training are shown in Table 1.

Some key hyperparameters (learning rate, discount factor, batch size) are initially set based on prior work and then fine-tuned through preliminary experiments. By performing local sensitivity analysis, each hyperparameter is varied individually within a reasonable range while keeping all other parameters fixed, in order to evaluate its impact on performance and stability. During this process, the stability of the average reward and network losses is monitored to identify robust configurations. The results demonstrate that the

Table 1 Related hyperparameters

Hyperparameters	Symbols	Values
Learning Rate	α	0.0003
Soft update coefficient	τ	0.0003
Batch Size	-	128
Buffer Size	-	1000000
Discount Factor	λ	0.99
Epsilon Decay	ε	0.9995
Policy Noise	-	0.2
Policy Noise Clip	-	0.5
Policy Update Frequency	-	2
Historical Information Length	T	10

proposed BMTD3 achieves more stable performance with a learning rate of 0.0003, a batch size of 128, and a discount factor of 0.99, confirming the reliability of the hyperparameter selection. As shown in Table 2.

5.2 Comparative Experiments

In mapless navigation, BMTD3 has long-term memory and hence has the capability of processing continuous historical inputs and dynamic obstacles. In this subsection, the performance of our BMTD3 method is further verified in comparison with GRU-Attention-TD3 [44], TD3 [21], LTD3(LSTM+TD3) [46], SAC-LSTM [47], and PPO-based navigation [48]. To this end, the training is conducted in Scenarios 1, 2, and 3 based on 1600, 2800, and 2000, respectively, and in Scenarios 4, 5, and 6 with 3500 episodes.

The performance analysis will be discussed from four aspects, that is, (1) the average reward, (2) the average actor and critic loss, (3) navigation trajectories successfully reached the target and their corresponding running times, (4) the total number of successful attempts to reach the target position, and (5) computational complexity and average inference time. It should be pointed out that the number

Table 2 Sensitivity analysis

Learning rate	Batch size	Discount factor	Performance & Stability
0.0003	128	0.99	↑
0.003	128	0.99	↓
0.03	128	0.99	↓
0.0003	256	0.99	↓
0.0003	512	0.99	↓
0.0003	128	1	↓
0.0003	128	0.96	↓

Note: ↑ means the improvement and ↓ stands for the deterioration

of episodes in each scenario is determined through empirical convergence analysis, where key performance indicators such as the average reward and the loss curves of the actor and critic networks are continuously monitored during training. Furthermore, to prevent overfitting to the specific training scenarios, random target conditions and dynamic obstacle trajectories are applied in each episode, and regularization mechanisms such as target network soft updates are used, ensuring stable learning and good generalization capability.

5.2.1 The Average Reward

The average reward is used to reflect the overall learning effect and performance of the robot in training, and to evaluate whether the policy is successful and whether it is superior to other methods. It is important to note that higher reward values lead to better navigation performance, as the robot can reach target positions more frequently and maintain stability. For presentation, the average rewards for BMTD3, TD3, and PPO are displayed in Fig. 5, while those for GRU-Attention-TD3, LTD3, and SAC-LSTM are shown in Fig. 6. In these two figures, each of the six lines shows the average reward of six different methods in six different scenarios, where the unit of the abscissa is 10 episodes and hence the ordinate is the average reward per 10 episodes.

Scenario 1 has mostly positive average rewards for BMTD3 and LTD3, depending on these two figures, but TD3, GRU-Attention-TD3, SAC-LSTM, and PPO have mostly negative average rewards. It should be pointed out that both BMTD3 and LTD3 significantly outperform GRU-Attention-TD3, TD3, SAC-LSTM, and PPO, and BMTD3 achieves a higher proportion of rewards above 2000 compared to LTD3. In what follows, BMTD3 shows a high proportion of positive average rewards under both Scenario 2 and Scenario 3, while the other five methods mostly yield negative rewards. Particularly, GRU-Attention-TD3 and LTD3 generally remain below -2000 . In Scenario 4, approximately half of BMTD3's average rewards are positive, exceeding those of the other five methods. Specifically, GRU-Attention-TD3, TD3, PPO, and SAC-LSTM show negative average rewards, with PPO and SAC-LSTM mostly below -2000 , while LTD3 has a small portion of positive rewards. BMTD3 achieves average rewards mostly above 1000 in both Scenario 5 and Scenario 6. In contrast, the performance of LTD3 is generally lower than -3000 , while other methods usually yield rewards below -2000 . The minimum average rewards of BMTD3 consistently surpass those of the other five methods in all six scenarios.

According to deep reinforcement learning, the low rewards of GRU-Attention-TD3, TD3, LTD3, SAC-LSTM, and PPO indicate that the robot might collide more easily, spin in place, locally rotate, or have timeouts. In comparison, BMTD3 has

the best performance, helping the robot navigate to the target position while avoiding obstacles. In these six scenarios, TD3, as a traditional method, is suitable for static obstacle environments with good general performance. Increased dynamic obstacles and complexity of the environment lead to a decrease in average reward and minimum average reward. Although GRU-Attention-TD3 can process historical inputs, it only performs well in static environments, and its average reward is negative in complex dynamic environments. Because of its ability to reduce collisions, the minimum average reward is larger than TD3. The LTD3 method with LSTM can handle static and dynamic obstacles to some extent, but it cannot handle historical inputs, so its average reward becomes worse when the number of dynamic obstacles and the complexity of the environment improve. Since it can effectively reduce collisions, the minimum average reward is larger than GRU-Attention-TD3. SAC-LSTM, a combination of SAC and LSTM, can model temporal dependencies in observations, but it is vulnerable to instability in highly dynamic environments, resulting in negative average rewards and poor stability. PPO is a common choice for robotic navigation tasks because of its stability in training, but it cannot adapt effectively to dynamic obstacles, leading to negative average rewards and frequent navigation failures. The BiGRU module in BMTD3 can deal with historical inputs and dynamic obstacles, and the MHA mechanism can further strengthen its focus on dynamic obstacles to effectively reduce collisions while reaching the target position. Therefore, BMTD3 has the best average reward and the best performance.

5.2.2 The Average Actor and Critic Loss

The experiments show that these approaches under the six scenarios are all convergent, and their average actor and critic loss in the final episode (i.e. approximate convergence values) is provided in Tables 3 and 4. For an episode, the loss functions of the actor and critic networks are calculated once at each step, are then accumulated and averaged at the end of the episode.

It follows from these two tables that the average actor loss of BMTD3 converges to 20 under simple Scenario 1, which is much larger than the other five methods. The average actor loss of BMTD3 in Scenario 2 is not significantly different from other methods. For Scenarios 3 and 4, the average actor losses of BMTD3 are 6.28 and 6.68, respectively, which are generally lower than those of the other methods. For Scenarios 5 and 6, they are 2.98 and 3.49, and they are smaller than those of the other methods. In summary, when the dynamic obstacles occur and the environment complexity increases, the average actor's loss of BMTD3 decreases, and the policy of BMTD3 is relatively stable in complex environments. In

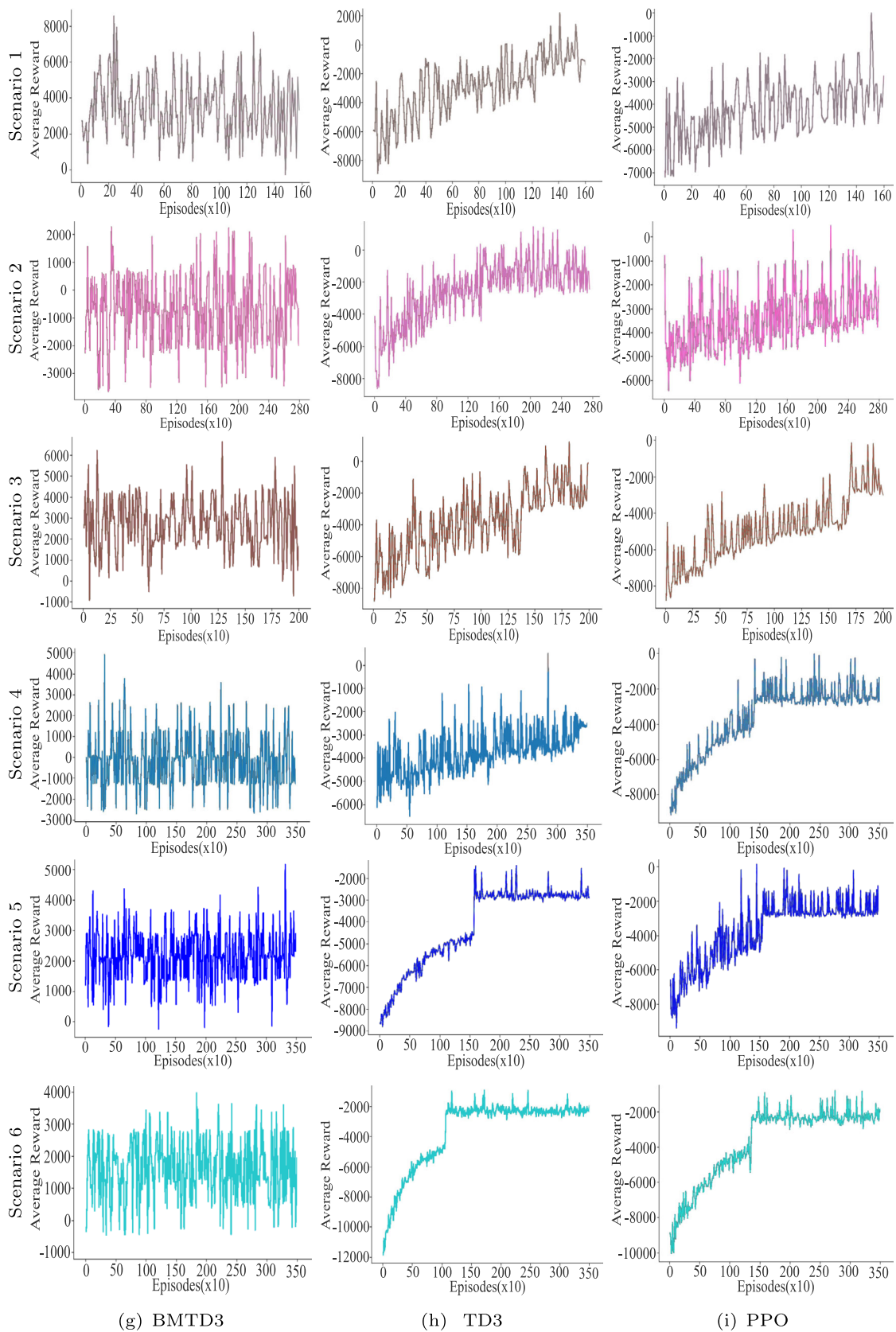


Fig. 5 The average reward calculated for every 10 episodes. The curves from the first to sixth rows correspond to the rewards of BMTD3, TD3, and PPO in Scenarios 1-6

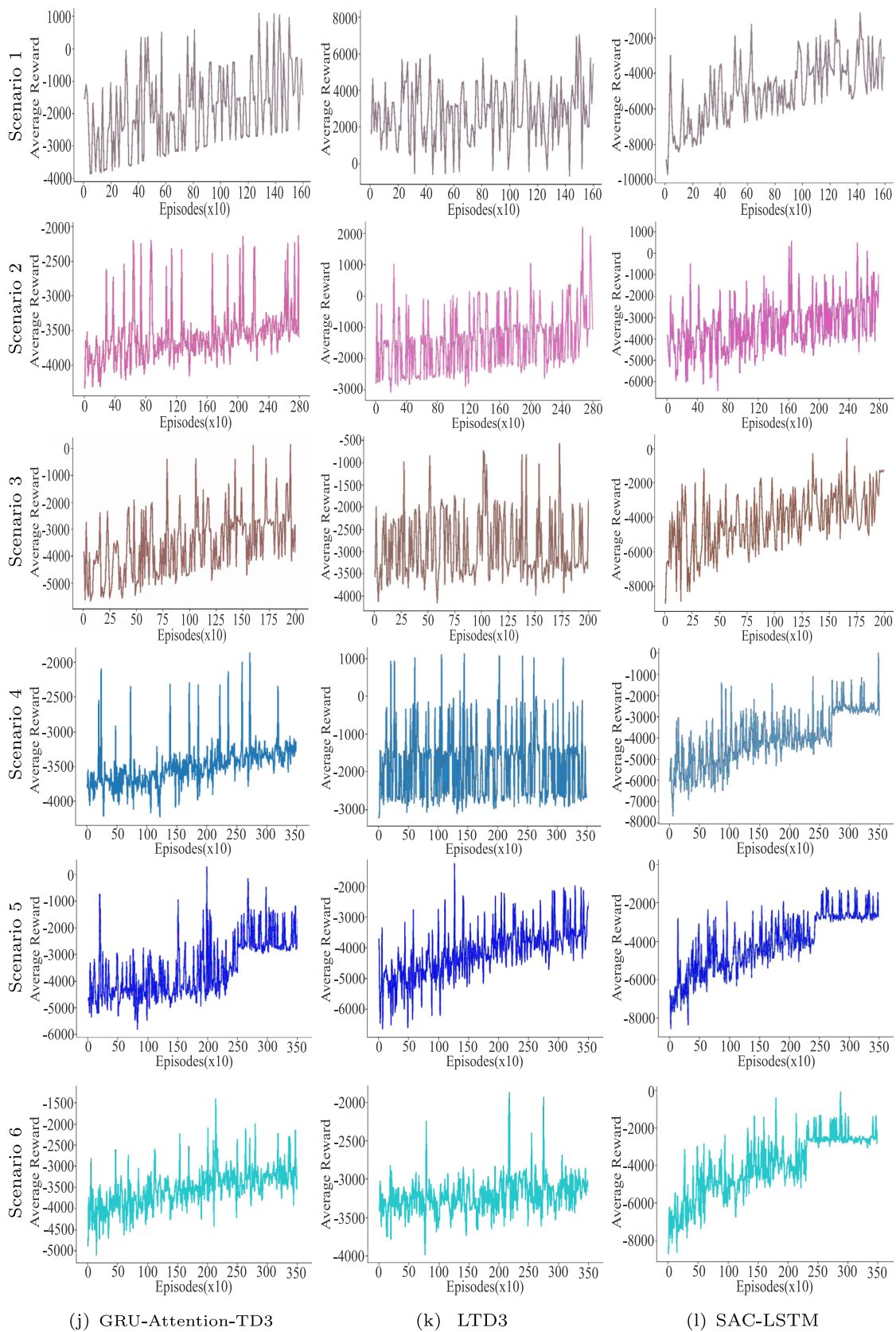


Fig. 6 The average reward calculated for every 10 episodes. The curves from the first to sixth rows correspond to the rewards of GRU-Attention-TD3, LTD3, and SAC-LSTM in Scenarios 1-6

Table 3 Average critic loss

Scenarios	TD3 [21]	LTD3 [46]	GRU-Attention-TD3 [44]	SAC-LSTM [47]	PPO [48]	BMTD3
1	18	56	12	22	22	28
2	19	18	39	28	30	47
3	22	27	24	24	22	51
4	34	61	33	30	25	90
5	28	42	35	33	30	80
6	26	40	36	35	23	50

what follows, the average critic losses of BMTD3 converge to 28, 47, 51, 90, 80, and 50, respectively, which are relatively larger than the other methods. It can be further found that, as the complexity of the environment increases, the average critic loss of BMTD3 increases. BMTD3's network structure is more complex, which carries out a more complex task of mapping observation-state-action values than other methods. This type of increased representation ability can provide more instructive feedback, thus supporting the stable optimization of the actor network.

5.2.3 Navigation trajectories successfully reached the target and their corresponding running times

The robot paths, successfully navigated to the target area, are shown in Fig. 7. The blue dot is the starting position of the robot, and the red cross mark is the position of the target area. As shown in Fig. 7a, the displayed trajectories correspond to the best-performing paths generated by each method. Although the goal positions differ across the methods, the distance from the robot's initial position to the target is approximately the same. It is evident that these existing methods often result in detours and frequent unnecessary turns, which reduce overall navigation efficiency. Specifically, TD3 exhibits local circling behavior, indicating instability in its policy execution. In contrast, our BMTD3 method generates shorter and more stable paths, with more direct and concise trajectories, fewer deviations, and smoother movements. In Fig. 7b, the navigation method based on BMTD3 makes the robot turn smoothly and navigate to the target area.

The other five methods have severe detour behavior, which is not conducive to navigation. In Fig. 7c, BMTD3 has a shorter path compared to GRU-Attention-TD3, and it is more direct to reach the target area. TD3, SAC-LSTM, and PPO exhibit local rotational behaviors, while LTD3 shows detouring behavior. In Fig. 7d, since the increase in environmental complexity has a great impact on the target areas navigated by the TD3, GRU-Attention-TD3 and LTD3 methods, the navigation paths with the same target area are selected for analysis. It can be seen that the navigation path of the BMTD3 method is shorter than that of the other three methods. In addition, SAC-LSTM and PPO display local rotational behaviors. In Fig. 7e, it is evident that all methods except BMTD3 demonstrate local rotational behaviors during navigation to the target, which adversely affect obstacle avoidance. In Fig. 7f, BMTD3 produces a direct and smooth trajectory, whereas the other five methods again show local rotational behaviors. In summary, TD3, SAC-LSTM, and PPO have the worst performance and are prone to local rotation. GRU-Attention-TD3 and LTD3 perform second and are prone to detours. Therefore, the navigation method based on BMTD3 has the best performance in the six scenarios. It has shorter and more direct paths, smoother and more continuous motion trajectories, and can reach the target areas more efficiently.

According to the robot navigation trajectories illustrated in Fig. 7, the time consumed by the robot to reach the target position is presented in Table 5. As shown in Fig. 7, although the target positions reached by the robot under different methods vary in each scenario, the distances between the starting and ending points differ only slightly. As shown in Table 5,

Table 4 Average actor loss

Scenarios	TD3 [21]	LTD3 [46]	GRU-Attention-TD3 [44]	SAC-LSTM [47]	PPO [48]	BMTD3
1	6.93	7.34	7.25	9.35	7.56	20
2	5.91	7.02	10.60	8.43	6.91	7.33
3	6.50	8.23	8.07	8.82	6.42	6.28
4	7.48	8.68	9.20	10.80	5.87	6.68
5	7.05	7.24	8.46	8.16	8.17	2.98
6	5.40	8.85	9.12	7.12	8.64	3.49

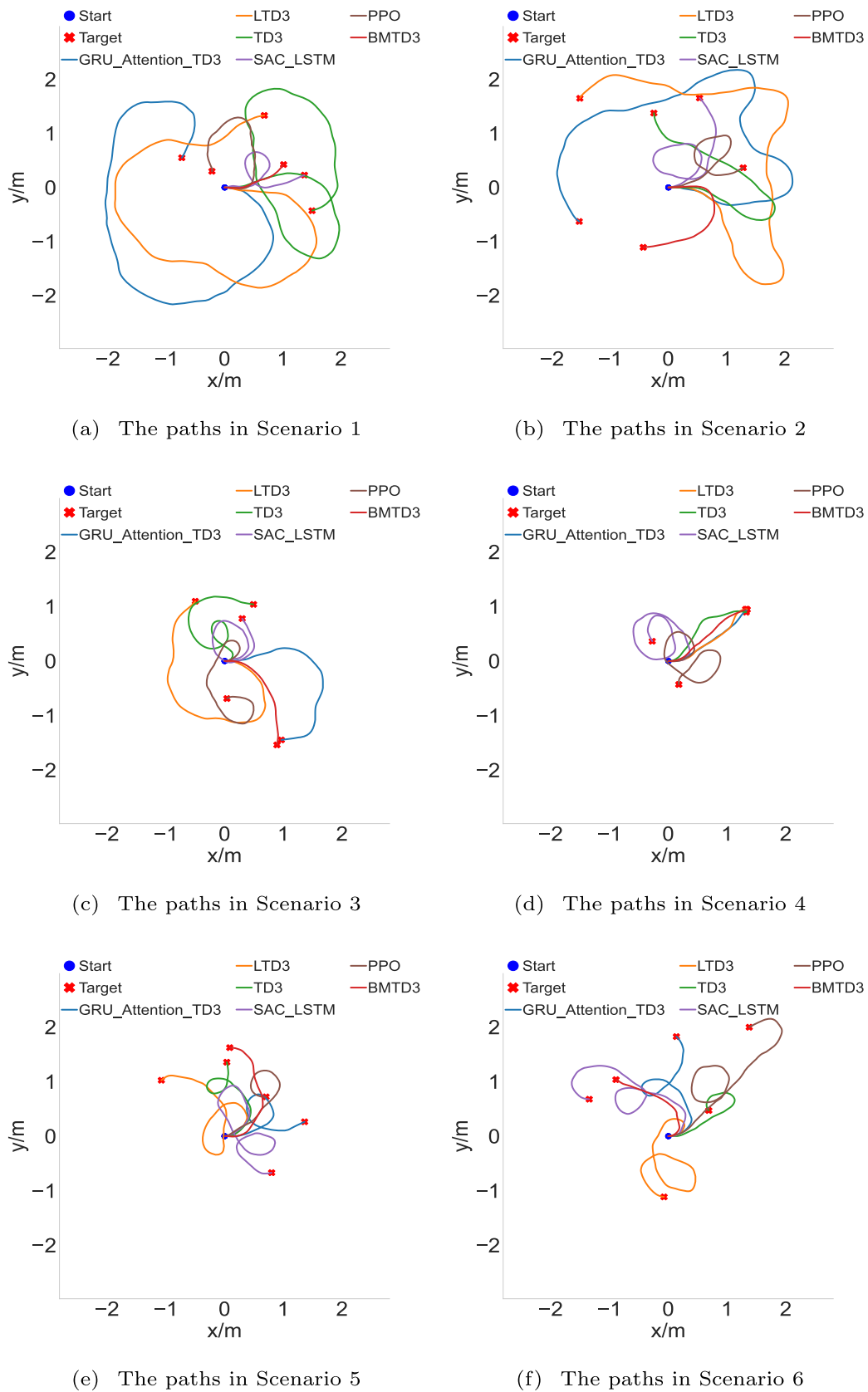


Fig. 7 Paths of the six methods in scenarios 1, 2, 3, 4, 5, and 6

Table 5 Running times of successful navigation trajectories

Scenarios	TD3 [21]	LTD3 [46]	GRU-Attention-TD3 [44]	SAC-LSTM [47]	PPO [48]	BMTD3
1	31.13	53.20	43.41	15.56	16.39	2.34
2	29.48	57.29	41.26	34.63	21.75	6.41
3	19.34	25.31	18.97	18.62	23.37	7.14
4	6.54	6.31	6.39	20.14	15.97	3.42
5	19.47	27.19	21.23	28.36	22.65	6.55
6	14.51	20.07	20.96	28.99	35.60	6.74

Note: In the table, the bold numbers in each scenario indicate the least time consumed during navigation, measured in seconds

the running times of BMTD3 for navigating to the target position in Scenarios 1, 2, 3, 4, 5, and 6 are 2.34, 6.41, 7.14, 3.42, 6.55, and 6.74 seconds, respectively. Compared with other navigation methods in the corresponding scenarios, BMTD3 achieves the lowest running time for successfully reaching the target position.

5.2.4 The Total Number of Successfully Reaching the Target Position

In six scenarios, the total number of successful attempts in reaching the target position is illustrated in Fig. 8. In Scenario 1, BMTD3 achieved 882 successful attempts, which is 44.63% higher than GRU-Attention-TD3, 41.31% higher than TD3, 8.56% higher than LTD3, 46.94% higher than SAC-LSTM, and 47.75% higher than PPO. Under Scenario 3, the total number of successful attempts for BMTD3 is 1127. Its success rate surpasses that of GRU-Attention-TD3 by 48%, TD3 by 45.56%, LTD3 by 51.75%, SAC-LSTM by 47.25%, and PPO by 52.95%. The total successful number for BMTD3 stands at 1411 in Scenario 5, showcasing a success rate that surpasses several other models. Specifically, it is 37.37% higher than GRU-Attention-TD3, 40.06% higher

than TD3, 39.06% higher than LTD3, 38.17% higher than SAC-LSTM, and 37.06% higher than PPO. In Scenario 6, the total number of BMTD3 is 1052, and the success rate exceeds GRU-Attention-TD3 by 29.31%, TD3 by 29.86%, LTD3 by 29.91%, SAC-LSTM by 27.34%, and PPO by 29.54%. According to the above analysis, it can conclude that as the complexity of the environment increases, especially in dynamic environments, the total number of successes of BMTD3 is much larger than that of GRU-Attention-TD3, TD3, LTD3, SAC-LSTM, and PPO. This indicates that BMTD3 has a higher success rate in map-free navigation, better navigation capabilities, and stronger adaptability in complex dynamic environments.

5.2.5 Computational Complexity and Average Inference Time

The merits of our method, BMTD3, are further explored in terms of computational complexity, model size, and average inference time per step. It is compared with several fashionable methods, such as GRU-Attention-TD3, TD3, LTD3, SAC-LSTM, and PPO, to highlight its advantages. The results are summarized in Table 6. In terms of this table,

Fig. 8 The total number of successfully reaching the target position

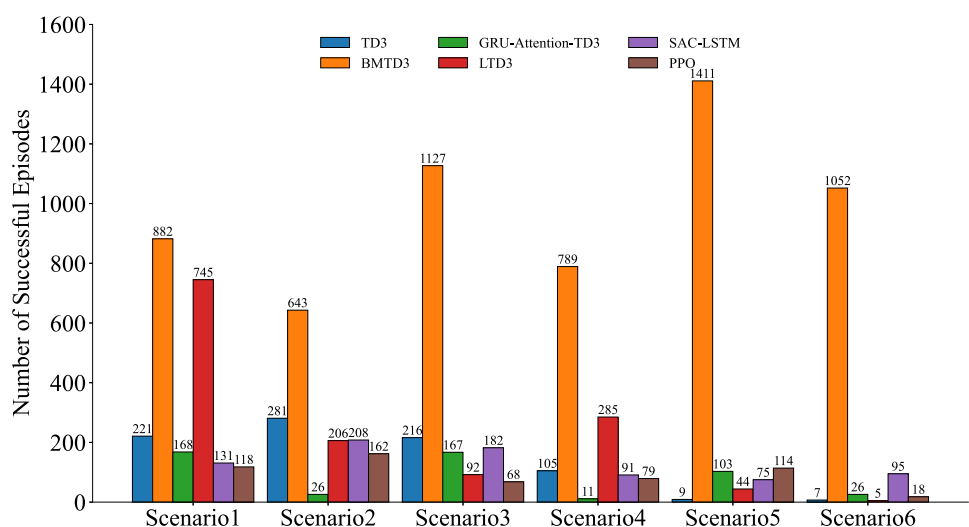


Table 6 Computational complexity, model size, and average inference time

Methods	FLOPs($\times 10^6$)	Parameters(M)	Avg Inference Time (s/step)
TD3 [21]	0.22	0.30	0.069
LTD3 [46]	12.1	3.5	0.142
GRU-Attention-TD3 [44]	8.73	2.4	0.109
SAC-LSTM [47]	4.0	1.2	0.038
PPO [48]	0.22	0.35	0.037
BMTD3	2.3	1.1	0.029

Note: Computational complexity is evaluated by the floating-point operations (FLOPs) required for one forward pass; Model size denotes the total number of learnable parameters; Avg inference time per step represents the average time (in seconds) required for one decision step.

the computational complexity of our method is higher than PPO and TD3, but it is just 19.01% of LTD3 and 26.36% of GRU-Attention-TD3. Furthermore, the model size of our method is also higher than PPO and TD3, but it is just 31.43% of LTD3 and 45.83% of GRU-Attention-TD3. It should be highlighted that our average inference time is the fastest, only 20.42% of LTD3, 42.03% of TD3, 76.32% of SAC-LSTM, 78.38% of PPO. Taking the success rate into account (see Fig. 8), although the number of parameters and computational complexity of our method are much higher than PPO, the inference time has slightly decreased, and the success rate has significantly improved.

Let us further analyze the mechanism of the above results. First, while TD3 and PPO demonstrate low computational complexity, they lack mechanisms for handling historical information and attention. This limitation hampers their navigation performance, often resulting in lower success rates, especially in complex situations. SAC-LSTM introduces LSTM to process historical information, improving performance at the cost of higher computational complexity. LTD3 and GRU-Attention-TD3 further enhance performance using attention mechanisms, which come with higher computational demands and longer inference times. In contrast, BMTD3 integrates BiGRU with MHA to capture historical dependencies effectively while maintaining relatively low

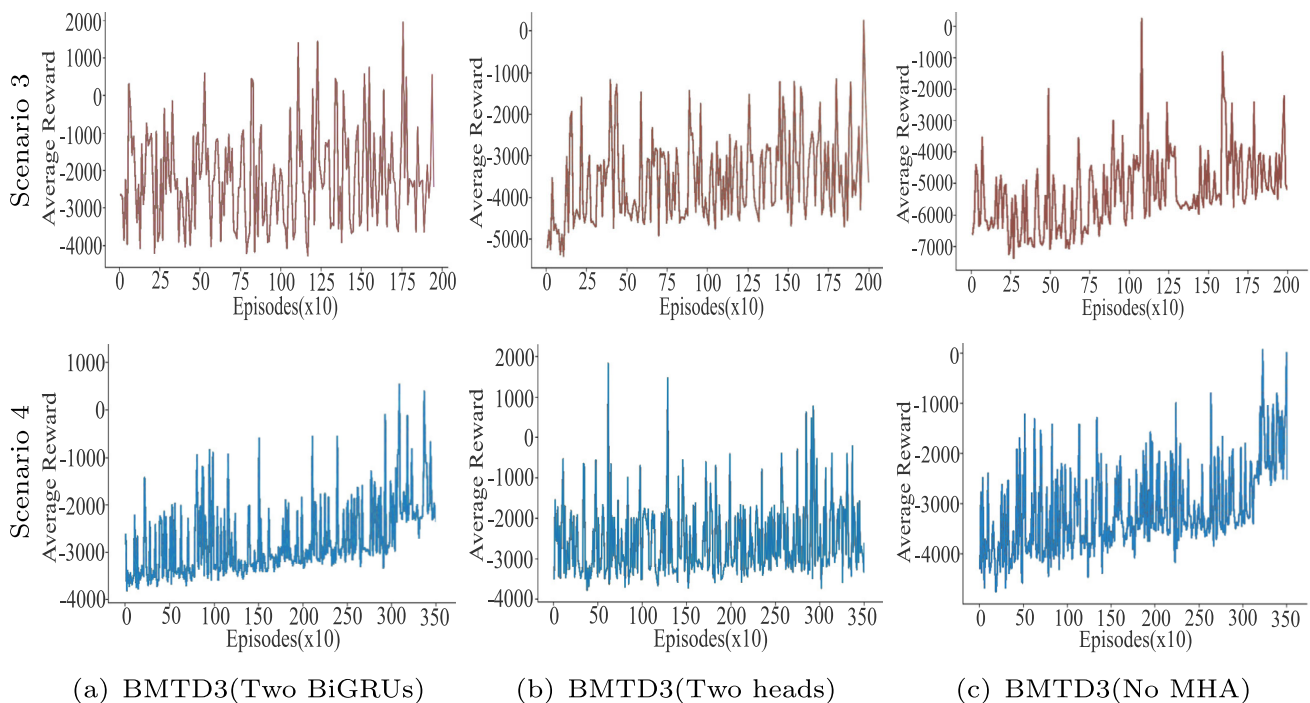


Fig. 9 The results of ablation experiments. The first and second rows display the average rewards achieved through training in Scenarios 3 and 4, respectively. The first column reflects the addition of one extra BiGRU layer, the second column shows the implementation of two

attention heads in BMTD3, and the third column illustrates the removal of the MHA module from BMTD3. The average reward is calculated for every 10 episodes

computational overhead. With only 2.3×10^6 FLOPs and 1.1M parameters, BMTD3 achieves the best overall navigation performance, realizing a superior balance between efficiency and effectiveness compared with all baselines.

5.3 Ablation Experiments

To further systematically evaluate the performance of the proposed BMTD3 method, ablation experiments are conducted by modifying or removing specific modules. Specifically, the experiment setting includes: (a) adding one additional BiGRU layer in BMTD3, (b) adopting 2 attention heads in BMTD3, and (c) removing the MHA mechanism. The experiments are conducted in both Scenario 3 and Scenario 4, with 2000 and 3500 episodes of training.

The results of the ablation studies are presented in Fig. 9. It follows from Fig. 9(a) that the average reward of BMTD3 with two stacked BiGRU layers in Scenario 3 remains largely negative, while its average reward is mostly below -2000 in Scenario 4. Moreover, the minimum average reward approaches -4000 in these two scenarios. In contrast, the average reward of the standard BMTD3 with a single BiGRU layer is consistently above 1000 in Scenario 3 and largely positive in Scenario 4, as shown in Fig. 5(a). It indicates that simply increasing the number of BiGRU layers does not improve and may even hinder the performance, due possibly to over-parameterization, and could reduce training stability. In light of Fig. 9(b), one can find that BMTD3 with two heads achieves an average reward of less than -1000 in Scenario 3 and less than -2000 in Scenario 4. In comparison with the results in Fig. 5(a) for BMTD3 with four attention heads, it can conclude that reducing the number of attention heads will lead to a negative effect. Finally, it follows from Fig. 9(c) that the average reward of BMTD3 without the MHA module is below -4000 in Scenario 3 and below -1000 in Scenario 4, respectively. It indicates that removing the MHA module substantially degrades the performance of BMTD3.

Table 7 presents the perception latency, communication latency, and actuation delay of BMTD3 under different levels of sensor noise. At a noise level of just 0.01, perception and communication latencies remain quite low, at 5 ms and 7 ms respectively, while the actuator delay remains around 12 ms, ensuring smooth real-time control. Under a moderate noise

level of 0.05, perception latency nearly doubles to 11 ms due to increased processing complexity, and the actuator delay rises to 18 ms, causing a slight reduction in system real-time performance. Under a severe noise level of 0.10, perception and communication latencies increase significantly to 18 ms and 16 ms, and the actuator delay reaches 27 ms, resulting in decreased responsiveness and instances of navigation failure. These results indicate that our BMTD3 framework maintains the feasible real-time performance under low or moderate noise conditions, while severe noise conditions should affect its stability.

6 Conclusion

This paper proposes a DRL-based mapless autonomous navigation framework, BMTD3, which addresses goal-driven navigation tasks in environments with both static and dynamic obstacles by integrating current observations with historical observation sequences. A task-adaptive BiGRU-MHA module is incorporated into both the actor and critic networks to extract features from the historical observation sequences. Specifically, the BiGRU endows the network with memory capabilities to alleviate partial observability issues and leverages its bidirectional structure to process historical observations and extract temporal features. The introduction of an MHA mechanism enables the extraction of distinct key historical features for policy generation and value estimation during navigation. Furthermore, the TD3 module is modified to allow the concatenation of extracted historical features. A novel reward function is also incorporated to improve navigation accuracy and adaptability. Simulation results demonstrate that, compared to traditional TD3 and other navigation methods, BMTD3 achieves higher navigation success rates and more effective navigation performance.

Although BMTD3 demonstrates superior performance, several failure cases emerge under extreme conditions. In dense dynamic obstacle scenarios, limited free space and unpredictable obstacle motions often cause delayed responses or collisions. When obstacles intersect near the target, BMTD3 may struggle to adapt in time, leading to oscillations or failures. In narrow passages formed by static and dynamic obstacles, the robot also tends to encounter difficul-

Table 7 Latencies and actuation delays under different levels of sensor noises

Noise levels	Noise variance	Perception latency(ms)	Communication latency(ms)	Actuation delay(ms)
Normal	0.01	5	7	12
Moderate	0.05	11	10	18
Severe	0.10	18	16	27

ties. These limitations suggest that, while effective overall, BMTD3 still faces challenges in highly dynamic environments. Future research will focus on further enhancing the robot's navigation capabilities in increasingly complex environments.

Author Contributions Haohua Zhang: Conceptualization, Methodology, Writing-original draft; Ying Sun: Supervision, Writing-review & editing, Funding; Pengyuan Liu: Investigation, Writing-review & editing; Derui Ding: Supervision, Writing-review & editing, Funding; and Ran Sun: Investigation.

Funding This work was supported in part by the National Natural Science Foundation of China under Grants 62203306 and 62373251.

Data Availability The data used in this paper are all publicly available and have no ethical violations.

Code Availability The code can be found from <https://github.com/zhanghnb/BMTD3/tree/master>.

Declarations

Conflicts of Interest Statement The authors declare no potential conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Kiran, B.R., Sobh, I., Talpaert, V., Mannion, P., Sallab, A.A.A., Yogamani, S., Pérez, P.: Deep reinforcement learning for autonomous driving: A survey. *IEEE Trans. Intell. Transp. Syst.* **23**(6), 4909–4926 (2022). <https://doi.org/10.1109/TITS.2021.3054625>
- Haydari, A., Yilmaz, Y.: Deep reinforcement learning for intelligent transportation systems: A survey. *IEEE Trans. Intell. Transp. Syst.* **23**(1), 11–32 (2022). <https://doi.org/10.1109/TITS.2020.3008612>
- Fu, G., Wang, Y., Yang, J.: Monocular visual navigation algorithm for nursing robots via deep learning oriented to dynamic object goal. *J. Intell. Robot. Syst.* **110**, 6 (2024). <https://doi.org/10.1007/s10846-023-02024-9>
- Le, H., Saeedvand, S., Hsu, C.C.: A comprehensive review of mobile robot navigation using deep reinforcement learning algorithms in crowded environments. *J. Intell. Robot. Syst.* **110**, 158 (2024). <https://doi.org/10.1007/s10846-024-02198-w>
- Niroui, F., Zhang, K., Kashino, Z., Nejat, G.: Deep reinforcement learning robot for search and rescue applications: Exploration in unknown cluttered environments. *IEEE Robot. Autom. Lett.* **4**(2), 610–617 (2019). <https://doi.org/10.1109/LRA.2019.2891991>
- Pore, A., Li, Z., Dall, D., Hernansanz, A., De, M.E., Menciassi, A., Casals, G.A., Dankelman, J., Fiorini, P., Poorten, E.V.: Autonomous navigation for robot-assisted intraluminal and endovascular procedures: A systematic review. *IEEE Trans. Robot.* **39**(4), 2529–2548 (2023). <https://doi.org/10.1109/TRO.2023.3269384>
- Lauri, M., Hsu, D., Pajarinen, J.: Partially observable markov decision processes in robotics: A survey. *IEEE Trans. Robot.* **39**(1), 21–40 (2023). <https://doi.org/10.1109/TRO.2022.3200138>
- Durrant-Whyte, H., Bailey, T.: Simultaneous localization and mapping: Part i. *IEEE Robot. Autom. Mag.* **13**(2), 99–110 (2006). <https://doi.org/10.1109/MRA.2006.1638022>
- Lv, X., He, Z., Yang, Y., Nie, J., Dong, Z., Wang, S., Gao, M.: MSF-SLAM: Multi-sensor-fusion-based simultaneous localization and mapping for complex dynamic environments. *IEEE Trans. Intell. Transp. Syst.* **25**(12), 19699–19713 (2024). <https://doi.org/10.1109/TITS.2024.3451996>
- Zhou, D., Wang, Z., Bandyopadhyay, S., Schwager, M.: Fast, on-line collision avoidance for dynamic vehicles using buffered voronoi cells. *IEEE Robot. Autom. Lett.* **2**(2), 1047–1054 (2017). <https://doi.org/10.1109/LRA.2017.2656241>
- Li, J., Dai, J., Su, Z.: RGB-D based visual SLAM algorithm for indoor crowd environment. *J. Intell. Robot. Syst.* **110**, 27 (2024). <https://doi.org/10.1007/s10846-023-02046-3>
- Yue, Y., Zhao, C., Wu, Z., Yang, C., Wang, Y., Wang, D.: Collaborative semantic understanding and mapping framework for autonomous systems. *IEEE/ASME Trans. Mechatron.* **26**(2), 978–989 (2021). <https://doi.org/10.1109/TMECH.2020.3015054>
- Dong, L., He, Z., Song, C., Sun, C.: A review of mobile robot motion planning methods: from classical motion planning workflows to reinforcement learning-based architectures. *J. Syst. Eng. Electron.* **34**(2), 439–459 (2023). <https://doi.org/10.23919/JSEE.2023.000051>
- Alali, M., Kazeminajafabadi, A., Imani, M.: Deep reinforcement learning sensor scheduling for effective monitoring of dynamical systems. *Syst. Sci. Control Eng.* **12**(1), Article 2329260 (2024). <https://doi.org/10.1080/21642583.2024.2329260>
- Wu, Y., Huang, X., Tian, Z., Yan, X., Yu, H.: Emotion contagion model for dynamical crowd path planning. *Int. J. Netw. Dyn. Intell.* **3**(3), 100014 (2024). <https://doi.org/10.53941/ijndi.2024.100014>
- Wang, X., Wang, S., Liang, X., Zhao, D., Huang, J., Xu, X., Dai, B., Miao, Q.: Deep reinforcement learning: A survey. *IEEE Trans. Neural Netw. Learn. Syst.* **35**(4), 5064–5078 (2024). <https://doi.org/10.1109/TNNLS.2022.3207346>
- Sun, Y., Chen, M., Peng, K., Wu, L., Liu, C.: Finite-time adaptive optimal control of uncertain strict-feedback nonlinear systems based on fuzzy observer and reinforcement learning. *Int. J. Syst. Sci.* **55**(8), 1553–1570 (2024). <https://doi.org/10.1080/00207721.2024.2312882>
- Sutton, R.S., Barto, A.G.: Reinforcement learning: An introduction. *IEEE Trans. Neural Netw.* **9**(5), 1054 (1998). <https://doi.org/10.1109/TNN.1998.712192>
- Wang, Y., Shen, C., Huang, J., Chen, H.: Model-free adaptive control for unmanned surface vessels: a literature review. *Syst. Sci. Control Eng.* **12**(1), (2024). <https://doi.org/10.1080/21642583.2024.2316170>
- Schuster, M., Paliwal, K.K.: Bidirectional recurrent neural networks. *IEEE Trans. Signal Process.* **45**(11), 2673–2681 (1997). <https://doi.org/10.1109/78.650093>

21. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: Proc. 35th Int. Conf. Mach. Learn. (PMLR). **80**, 1587–1596 (2018). <https://proceedings.mlr.press/v80/fujimoto18a.html>
22. Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M.: Playing atari with deep reinforcement learning. *arXiv:1312.5602*. (2013). <https://doi.org/10.48550/arXiv.1312.5602>
23. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G.: Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015). <https://doi.org/10.1038/nature14236>
24. Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double q-learning. *Proc. AAAI Conf. Artif. Intell.* **30**(1), 2094–2100 (2016). <https://doi.org/10.1609/aaai.v30i1.10295>
25. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M.: Dueling network architectures for deep reinforcement learning. In: Proc. 33rd Int. Conf. Mach. Learn. (PMLR). **48**, 1995–2003 (2016). <https://proceedings.mlr.press/v48/wangf16.html>
26. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning. *arXiv:1509.02971*. (2019). <https://doi.org/10.48550/arXiv.1509.02971>
27. Tai, L., Paolo, G., Liu, M.: Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation. In: Proc. 2017 IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS). 31–36, (2017). <https://doi.org/10.1109/IROS.2017.8202134>
28. Mnih, V., Puigdomènech, B.A., Mirza, M., Graves, A., Lillicrap, T.P., Harley, T., Silver, D., Kavukcuoglu, K.: Asynchronous methods for deep reinforcement learning. In: Proc. 33rd Int. Conf. Mach. Learn. (PMLR). **48**, 1928–1937 (2016). <https://proceedings.mlr.press/v48/mniha16.html>
29. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. *arXiv:1707.06347*. (2017). <https://doi.org/10.48550/arXiv.1707.06347>
30. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Proc. 35th Int. Conf. Mach. Learn. (PMLR). **80**, 1861–1870 (2018). <https://proceedings.mlr.press/v80/haarnoja18b.html>
31. Hausknecht, M., Stone, P.: Deep recurrent q-learning for partially observable mdps. *arXiv:1507.06527*. (2015). <https://doi.org/10.48550/arXiv.1507.06527>
32. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. *Adv. Neural Inf. Process. Syst.* **30** (2017). https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf
33. Zhang, W., Liu, N., Zhang, Y.: Learn to navigate maplessly with varied LiDAR configurations: A support point-based approach. *IEEE Robot. Autom. Lett.* **6**(2), 1918–1925 (2021). <https://doi.org/10.1109/LRA.2021.3061305>
34. Shi, H., Shi, L., Xu, M., Hwang, K.S.: End-to-end navigation strategy with deep reinforcement learning for mobile robots. *IEEE Trans. Ind. Inform.* **16**(4), 2393–2402 (2020). <https://doi.org/10.1109/TII.2019.2936167>
35. Jin, J., Nguyen, N.M., Sakib, N., Graves, D., Yao, H., Jagerand, M.: Mapless navigation among dynamics with social-safety-awareness: A reinforcement learning approach from 2D laser scans. In: Proc. IEEE Int. Conf. Robot. Autom. (ICRA). 6979–6985 (2020). <https://doi.org/10.1109/ICRA40945.2020.9197148>
36. Zhang, W., Zhang, Y., Liu, N., Ren, K., Wang, P.: IPAPRec: A promising tool for learning high-performance mapless navigation skills with deep reinforcement learning. *IEEE/ASME Trans. Mechatron.* **27**(6), 5451–5461 (2022). <https://doi.org/10.1109/TMECH.2022.3182427>
37. Zhu, W., Hayashibe, M.: Learn to navigate in dynamic environments with normalized LiDAR scans. In: Proc. IEEE Int. Conf. Robot. Autom. (ICRA). 7568–7575 (2024). <https://doi.org/10.1109/ICRA57147.2024.10611247>
38. Everett, M., Chen, Y.F., How, J.P.: Motion planning among dynamic, decision-making agents with deep reinforcement learning. In: Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS). 3052–3059 (2018). <https://doi.org/10.1109/IROS.2018.8593871>
39. Xue, Y., Chen, W.: Combining motion planner and deep reinforcement learning for UAV navigation in unknown environment. *IEEE Robot. Autom. Lett.* **9**(1), 635–642 (2024). <https://doi.org/10.1109/LRA.2023.3334978>
40. Zhu, W., Hayashibe, M.: Autonomous navigation system in pedestrian scenarios using a dreamer-based motion planner. *IEEE Robot. Autom. Lett.* **8**(6), 3836–3843 (2023). <https://doi.org/10.1109/LRA.2023.3273514>
41. Han, Y., Zhan, I.H., Zhao, W., Pan, J., Zhang, Z., Wang, Y., Liu, Y.J.: Deep reinforcement learning for robot collision avoidance with self-state-attention and sensor fusion. *IEEE Robot. Autom. Lett.* **7**(3), 6886–6893 (2022). <https://doi.org/10.1109/LRA.2022.3178791>
42. Yang, B., Yuan, M., Zhang, C., Hong, C., Pan, G., Tang, H.: Spiking reinforcement learning with memory ability for mapless navigation. In: Proc. 2023 IEEE/RSJ Int. Conf. Intell. Robot. Syst. (IROS). 1–8 (2023). <https://doi.org/10.1109/IROS55552.2023.10341738>
43. Böhm, P., Pounds, P., Chapman, A.C.: Feature extraction for effective and efficient deep reinforcement learning on real robotic platforms. In: Proc. 2023 IEEE Int. Conf. Robot. Autom. (ICRA). 7126–7132 (2023). <https://doi.org/10.1109/ICRA48891.2023.10160862>
44. Jia, J., Xing, X., Chang, D.E.: GRU-attention based TD3 network for mobile robot navigation. In: Proc. 2022 22nd Int. Conf. Control, Autom. Syst. (ICCAS). 1642–1647 (2022). <https://doi.org/10.23919/ICCAS55662.2022.10003950>
45. Zhang, L., Song, Q., Fan, M., Yao, X.: Credit strategy research based on markov random field and enterprise association knowledge graph. *Int. J. Syst. Sci.* **55**(5), 844–857 (2024). <https://doi.org/10.1080/00207721.2023.2300149>
46. Ma, H., Wang, S., Zhang, S., Ren, S., Wang, H.: Map-less end-to-end navigation of mobile robots via deep reinforcement learning. In: Proc. 2023 IEEE 13th Int. Conf. Electr. Inf. Emerg. Comm. (ICEIEC). 48–52 (2023). <https://doi.org/10.1109/ICEIEC58029.2023.10200799>
47. Grando, R.B., de Jesus, J.C., Kich, V.A., Kolling, A.H., Guerra, R.S., Drews-Jr, P.L.J.: Deterministic and stochastic analysis of deep reinforcement learning for low dimensional sensing-based navigation of mobile robots. *Proc. Latin Amer. Robot. Symp. Braz. Symp. Robot. Workshop Robot. Educ.* 193–198 (2022). <https://doi.org/10.1109/LARS/SBR/WRE56824.2022.9995792>
48. Taheri, H., Hosseini, S.R., Nekoui, M.A.: Deep reinforcement learning with enhanced PPO for safe mobile robot navigation. *arXiv:2405.16266*. (2024). <https://doi.org/10.48550/arXiv.2405.16266>

Haohua Zhang received the B.Sc. degree in Automation from Jiangxi University of Science and Technology, Ganzhou, China, in 2023. He is currently working toward the M.Sc. degree in Control Science and Engineering at the University of Shanghai for Science and Technology, Shanghai, China. His research interests include autonomous mobile robot and deep reinforcement learning.

Ying Sun received the B.Sc. degree in Physics from Harbin Normal University, Harbin, China, in 2013. Earned the PhD in Control Science and Engineering in 2021, School of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology. From April 2021 to April 2024, she worked at the Postdoctoral Research Station of Systems Science at the Business School, University of Shanghai for Science and Technology. Since April 2024, she has been serving as a lecturer in the Department of Systems Science at the Business School, University of Shanghai for Science and Technology, Shanghai, China. She is an active reviewer for many international journals. Her current research interests include networked control systems, stochastic control and filtering as well as H_∞ , $l_2 - l_\infty$ control and filtering.

Pengyuan Liu received the B.Sc. degree in automatic from Suzhou University of Technology, Suzhou, China, in 2023. He is currently working toward the M.Sc. degree in Control Science and Engineering from University of Shanghai for Science and Technology, Shanghai, China. His research interests include networked control systems and their applications in automated vehicles.

Derui Ding received both the B.Sc. degree in Industry Engineering in 2004 and the M.Sc. degree in Detection Technology and Automation Equipment in 2007 from Anhui Polytechnic University, Wuhu, China, and the Ph.D. degree in Control Theory and Control Engineering in 2014 from Donghua University, Shanghai, China. He is currently a Senior Research Fellow with the School of Science, Computing and Engineering Technologies, Swinburne University of Technology, Melbourne, VIC, Australia. His research interests include distributed control, filtering and optimization, learning-based control as well as data mining.

Ran Sun received his Ph.D. degree in control science and engineering with the specialization in guidance and control from Shanghai Jiao Tong University, Shanghai, China, in 2019. He is currently a lecturer in school of Optical-Electrical and Computer Engineering, University of Shanghai for Science and Technology, Shanghai, China. His research interests include autonomous mobile robot multi-agent systems, and reinforcement learning.